Built-in tools and structured output fundamentals

0:45 / 12:05 2x

Connecting large language models to the real world

Large language models like Gemini are incredibly powerful. They can write stories, summarize articles, and answer complex questions based on the vast amount of information they were trained on. However, this training data has a limit. An AI model does not know what happened in the news this morning, the current price of a product on a shopping website, or the details of a technical document published yesterday. Its knowledge is frozen at the time of its last training.

To build helpful and intelligent applications, developers need a way to connect the model to the internet. This is where built-in tools come in, acting as the model's eyes and ears on the web. However, simply getting information from the internet is only half the battle. The information often comes back as a long, unstructured block of text—easy for a person to read but very difficult for a computer program to understand and use.

For an AI application to perform tasks reliably, it needs data that is clean, predictable, and organized. This is the challenge that structured output solves. By combining tools to gather real-time information with a system for structuring that information, developers can build sophisticated AI applications that are not only knowledgeable but also dependable. This article will explore two of Gemini's essential built-in tools—URL context and Google Search—and explain how to use JSON schema to create the structured output necessary for building modern AI agents.

Giving your AI access to the live web

To overcome the limitation of static training data, the Gemini API includes built-in tools that act as a bridge to external information. These tools allow the AI model to access and process information from the internet in real time, making your applications more accurate, relevant, and powerful. The two tools we'll cover in this module are URL context and Google Search. Each serves a different purpose and has unique strengths and weaknesses.

While this module focuses on URL context and Google Search, it is important to note that the Gemini API supports a broader ecosystem of tools. These advanced capabilities include connecting Gemini to external services like Google Maps, or providing the model with a secure environment to execute code for complex problem-solving. These advanced tools offer even more powerful ways to extend Gemini's functionality, but they are out of scope for this introductory module.

The URL context tool

The URL context tool gives Gemini the ability to "read" the content of a specific webpage. When you provide a web address (a URL) in your API request, the tool fetches the content from that page and provides it to the model as context for its answer. This is incredibly useful when you need the AI to work with information from a known source.

For example, imagine you are building an application for a financial analyst who needs to quickly understand a company's latest quarterly earnings report. Without the URL context tool, the analyst would have to copy and paste the entire report into the prompt. With the tool, the developer can simply provide the URL to the report. The AI can then read the content and perform tasks like summarizing the key findings, extracting specific financial numbers, or answering questions about the company's performance.

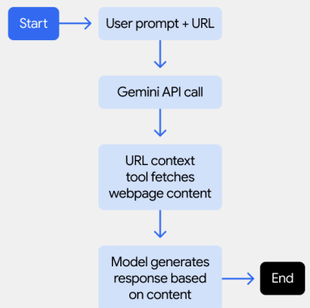
Similarly, a customer support team can build an internal tool that answers questions about their company's complex, frequently updated product documentation. By using the URL context tool, they can build a chatbot that always has the most current information. When an employee asks, "What are the steps to configure the new security feature?", the application can provide the URL of the relevant documentation page, and Gemini will provide a clear, step-by-step answer based on the official source.

However, the URL context tool has important limitations.

- **Access restrictions:** It cannot access every page. Many websites use a robots.txt file to block automated systems.
- **Protected content:** It cannot bypass paywalls or login screens where valuable content is often located.
- **Dynamic content:** Some modern websites rely heavily on JavaScript to load their content; if the information is not present in the initial HTML, the tool might not be able to see it.

Because of these limitations, developers must design their applications with backups in place and should not rely on this tool for information that is absolutely indispensable and may be protected.

URL context process



The Google search tool

While URL context is perfect for known sources, what if you need information about a current event or a topic you don't have a specific link for? This is where the Google Search tool becomes essential. With this tool, a practice often called "grounding," connects the Gemini model to Google Search. When the model needs information it doesn't have, it can perform a search, review the results, and use that information to form a more accurate and timely response. This is vital for applications that deal with news, current events, or fact-checking. The model's response is "grounded" in the information retrieved from the sources.

For instance, a travel planning application could use Google Search to answer a user's question like, "What are some upcoming festivals in Tokyo this month?" The model's training data would be out of date, but with Google Search, it can find current event listings and provide a relevant, helpful answer. Likewise, an application designed to combat misinformation could use the tool to fact-check a claim. If a user asks, "Is it true that a new law was just passed?", the model can use Google Search to find reliable news sources and report on the current legislative status.

The main benefit of Google Search "grounding" is access to a nearly limitless and constantly updated source of information. However, this power comes with trade-offs:

- **Latency:** Using Google Search can increase the time it takes for the API to return a response, as the model has to perform the search and process the results.
- **Cost:** API call costs may increase.
- **Variability:** The quality of the output depends on the quality of the search results. A search can sometimes return irrelevant or low-quality information, which could negatively affect the model's final response.

Developers must weigh the need for real-time information against these potential drawbacks.

Creating structured output with JSON Schema

Getting information from the web with tools is a huge step forward, but it creates a new problem: the output from the model is often a natural language paragraph. Let's return to the financial analyst example. After reading the earnings report, the model might return a response like this:

"The company reported strong quarterly results, with revenues of $1.2 billion, which is an increase of 15% from last year. Net income was $150 million, and their earnings per share (EPS) was $1.25. The outlook for next quarter is positive."

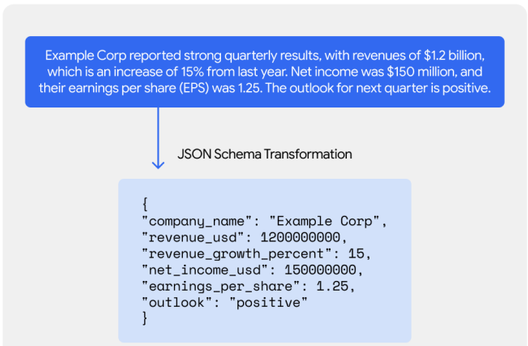
This is perfectly understandable for a human, but for the application's software, it's just a string of text. If the software needs to display the revenue in a dashboard or save the net income to a database, it would need complex code to "parse" this sentence. This process is fragile and can easily break if the model phrases its response differently next time.

This is where structured output becomes indispensable. Structured output forces the model to provide its answer in a consistent, machine-readable format like JSON (JavaScript Object Notation). The same financial data, structured as JSON, would look like this:

```
{
  "company_name": "Example Corp",
  "revenue_usd": 1200000000,
  "revenue_growth_percent": 15,
  "net_income_usd": 15000000,
  "earnings_per_share": 1.25,
  "outlook": "positive"
}
```

This format is predictable and easy for any programming language to understand. To get the model to use this exact format, use a JSON Schema—a blueprint that defines the exact structure and data types you expect. You provide this schema in your API call (using `response_mime_type` and `response_json_schema`), and Gemini generates syntactically valid JSON that matches that schema. The API enforces a JSON Schema–compatible subset, so you still need to validate the semantic correctness of the values in your own code. This transforms the model's response from a helpful but unstructured paragraph into a reliable data source that an AI agent can act upon. For an AI system to be considered an "agent," it must be able to perform actions. Structured output unlocks this capability by providing the clean data the agent needs to make decisions and interact with other software.

JSON Schema transformation



By defining a schema, you gain control and predictability. You can specify that revenue_usd must be a number, that outlook must be a string, and that both fields are required. This validation is a powerful feature for building robust applications. It reduces errors, simplifies your code, and builds a foundation of trust between the AI model and the rest of your system.

Key takeaways

- Large language models are limited by their static training data and require access to real-time information to be effective in modern applications.
- Built-in tools like URL Context and Google Search connect the model to the live internet, allowing it to work with up-to-date or specific external information.
- Raw information from these tools is often unstructured. Using a JSON Schema to enforce structured output transforms natural language responses into clean, predictable, machine-readable data.
- The combination of real-time data retrieval and structured output is the fundamental building block for creating sophisticated and autonomous AI agents that can analyze information and take meaningful action.

Go to next item Completed